



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

3 апреля 2025 г.

Цель работы

- Реализовать алгоритм растровой развертки многоугольника (в соответствии с вариантом): построчного заполнения с затравкой для восьмисвязной гранично-определенной области;
- Реализовать алгоритм фильтрации (в соответствии с вариантом): целочисленный алгоритм Брезенхема с устранением ступенчатости;
- Реализовать необходимые вспомогательные алгоритмы (растеризации отрезка) с модификациями, обеспечивающими корректную работу основного алгоритма: целочисленный алгоритм Брезенхема
- Ввод исходных данных каждого из алгоритмов производится интерактивно с помощью клавиатуры и/или мыши. Предусмотреть также возможность очистки области вывода (отмены ввода).
- Растеризацию производить в специально выделенном для этого буфере в памяти с последующим копированием результата в буфер кадра OpenGL. Предусмотреть возможность изменение размеров окна.

Теория

В данной работе реализованы растровые алгоритмы построения и фильтрации многоугольников с использованием целочисленных методов. На деле происходит отрисовка замкнутых фигур, их заливка и устранение ступенчатости линий.

В работе реализован метод построчного заполнения с затравкой (seed fill) для восьмисвязной гранично-определённой области. Это означает, что пиксели границы многоугольника считаются связанными, если они имеют общую сторону или вершину. Затравочный алгоритм начинается с одной внутренней точки (затравки), от которой заполняется вся внутренняя область, пока не будет достигнута граница, определённая цветом. Метод flood fill реализован итеративно с использованием стека, что позволяет избежать переполнения стека вызовов, типичного для рекурсивных реализаций. Алгоритм сначала расширяется по го-

ризонтали, находя на каждом шаге левую и правую границу области, а затем добавляет в стек пиксели строк выше и ниже для дальнейшего заполнения.

Границы многоугольника строятся с помощью целочисленного алгоритма Брезенхема. Для устранения эффекта ступенчатости (aliasing) используется модифицированный вариант алгоритма Брезенхема с применением сглаживания (антиалиасинга). Метод заключается в том, что интенсивность (альфа-канал) пикселя рассчитывается на основе расстояния до идеальной линии, что создаёт эффект более плавного визуального перехода между пикселями. Это особенно важно для повышения качества визуализации и делает контуры объектов менее "рваными" на глаз.

Реализация

Задание реализовано на питоне с использованием библиотек GLFW, OpenGL и NumPy. Отрисовка осуществляется в растровом буфере, представленном как трёхмерный массив NumPy, содержащий значения RGB для каждого пикселя.

Пользователь с помощью мыши задаёт вершины многоугольника. Затем между вершинами строятся линии, а потом происходит заливка, а также сглаживание ребер.

Графика отображается через OpenGL функцией `glDrawPixels`. При изменении размера окна автоматически масштабируются как буфер, так и координаты вершин.

Код

Листинг 1: Файл `main.py`

```
1 import glfw
2 from OpenGL.GL import *
3 import numpy as np
4
5 # Параметры окна
6 WIDTH, HEIGHT = 800, 800
7 framebuffer = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)
8 polygon = []
9
10 # Цвета
```

```

11 COLOR_WHITE = (255, 255, 255)
12 COLOR_POLYGON_FILL = (255, 0, 0)
13
14 def plot(x, y, color, alpha=1.0):
15     if 0 <= x < WIDTH and 0 <= y < HEIGHT:
16         framebuffer[y, x] = (framebuffer[y, x] * (1 - alpha) +
17                               np.array(color) * alpha).astype(np.uint8)
18
19 def draw_line(x0, y0, x1, y1, color):
20     dx, sx = abs(x1 - x0), 1 if x0 < x1 else -1
21     dy, sy = -abs(y1 - y0), 1 if y0 < y1 else -1
22     error = dx + dy
23     while True:
24         plot(x0, y0, color)
25         if x0 == x1 and y0 == y1:
26             break
27         e2 = 2 * error
28         if e2 >= dy:
29             error += dy
30             x0 += sx
31         if e2 <= dx:
32             error += dx
33             y0 += sy
34
35 def draw_circle(x1, y1, radius, color):
36     x, y = 0, radius
37     delta = 1 - 2 * radius
38     while y >= x:
39         for dx, dy in [(x, y), (x, -y), (-x, y), (-x, -y), (y, x), (y, -
40 x), (-y, x), (-y, -x)]:
41             plot(x1 + dx, y1 + dy, color)
42             error = 2 * (delta + y) - 1
43             if delta < 0 and error <= 0:
44                 delta += 2 * x + 1
45                 x += 1
46             elif delta > 0 and error > 0:
47                 delta -= 2 * y + 1
48                 y -= 1
49             else:
50                 delta += 2 * (x - y)
51                 x += 1
52                 y -= 1
53
54 def flood_fill(x, y, fill_color, boundary_color):
55     stack = [(x, y)]
56     while stack:

```

```

56         x, y = stack.pop()
57         if 0 <= x < WIDTH and 0 <= y < HEIGHT and \
58             not np.array_equal(framebuffer[y, x], fill_color) and \
59             not np.array_equal(framebuffer[y, x], boundary_color):
60             lx, rx = x, x
61             while lx > 0 and not np.array_equal(framebuffer[y, lx - 1],
boundary_color):
62                 lx -= 1
63                 while rx < WIDTH - 1 and not np.array_equal(framebuffer[y,
rx + 1], boundary_color):
64                     rx += 1
65                 for i in range(lx, rx + 1):
66                     plot(i, y, fill_color)
67                     for dy in [-1, 1]:
68                         ny = y + dy
69                         if 0 <= ny < HEIGHT and not np.array_equal(
framebuffer[ny, i], boundary_color):
70                             stack.append((i, ny))
71
72 def display():
73     glClear(GL_COLOR_BUFFER_BIT)
74     w, h = glfw.get_framebuffer_size(window)
75     glViewport(0, 0, w, h)
76     glDrawPixels(WIDTH, HEIGHT, GL_RGB, GL_UNSIGNED_BYTE, framebuffer)
77     glfw.swap_buffers(window)
78
79 def framebuffer_size_callback(window, new_width, new_height):
80     global WIDTH, HEIGHT, framebuffer, polygon
81     WIDTH, HEIGHT = new_width, new_height
82     framebuffer = np.resize(framebuffer, (HEIGHT, WIDTH, 3))
83     glViewport(0, 0, WIDTH, HEIGHT)
84
85 def calculate_centroid(polygon):
86     x_coords = [p[0] for p in polygon]
87     y_coords = [p[1] for p in polygon]
88     return sum(x_coords) // len(polygon), sum(y_coords) // len(polygon)
89
90 def draw_filtered_line(x0, y0, x1, y1, color):
91     deltax, deltay = abs(x1 - x0), abs(y1 - y0)
92     error = 0
93     deltaerr = max(deltax, deltay) + 1
94     x, y = x0, y0
95     dirx = 1 if x1 > x0 else -1 if x1 < x0 else 0
96     diry = 1 if y1 > y0 else -1 if y1 < y0 else 0
97
98     if deltax >= deltay:

```

```

99         for _ in range(deltax + 1):
100             alpha = 1 - (error / deltaerr)
101             plot(x, y, color, alpha)
102             x += dirx
103             error += deltay
104             if error >= deltaerr:
105                 y += diry
106                 error -= deltaerr
107         else:
108             for _ in range(deltay + 1):
109                 alpha = 1 - (error / deltaerr)
110                 plot(x, y, color, alpha)
111                 y += diry
112                 error += deltax
113                 if error >= deltaerr:
114                     x += dirx
115                     error -= deltaerr
116
117 def draw_polygon(polygon, color):
118     if len(polygon) > 2:
119         for i in range(len(polygon)):
120             draw_line(*polygon[i], *polygon[(i + 1) % len(polygon)],
121 COLOR_WHITE)
122             seed_x, seed_y = calculate_centroid(polygon)
123             flood_fill(seed_x, seed_y, color, COLOR_WHITE)
124             for i in range(len(polygon)):
125                 draw_filtered_line(*polygon[i], *polygon[(i + 1) % len(
126 polygon)], COLOR_WHITE)
127
128 def mouse_button_callback(window, button, action, mods):
129     global polygon
130     if action == glfw.PRESS:
131         x, y = glfw.get_cursor_pos(window)
132         x, y = int(x), HEIGHT - int(y)
133         if button == glfw.MOUSE_BUTTON_LEFT:
134             polygon.append((x, y))
135             draw_circle(x, y, 5, COLOR_WHITE)
136         elif button == glfw.MOUSE_BUTTON_RIGHT:
137             draw_polygon(polygon, COLOR_POLYGON_FILL)
138
139 def key_callback(window, key, scancode, action, mods):
140     global framebuffer, polygon
141     if key == glfw.KEY_R and action == glfw.PRESS:
142         framebuffer = np.zeros((HEIGHT, WIDTH, 3), dtype=np.uint8)
143         polygon = []

```

```

143 if __name__ == "__main__":
144     if not glfw.init():
145         exit()
146     window = glfw.create_window(400, 400, "Polygon Rasterization", None,
        None)
147     glfw.make_context_current(window)
148     WIDTH, HEIGHT = glfw.get_framebuffer_size(window)
149     framebuffer_size_callback(window, WIDTH, HEIGHT)
150     glfw.set_framebuffer_size_callback(window, framebuffer_size_callback
        )
151     glfw.set_mouse_button_callback(window, mouse_button_callback)
152     glfw.set_key_callback(window, key_callback)
153
154     while not glfw.window_should_close(window):
155         display()
156         glfw.poll_events()
157     glfw.terminate()

```

Вывод программы

Программа создала окно, после кликов левой кнопкой мыши появились точки-вершины многоугольника. А после нажатия правой кнопки мыши, многоугольник залился красным.

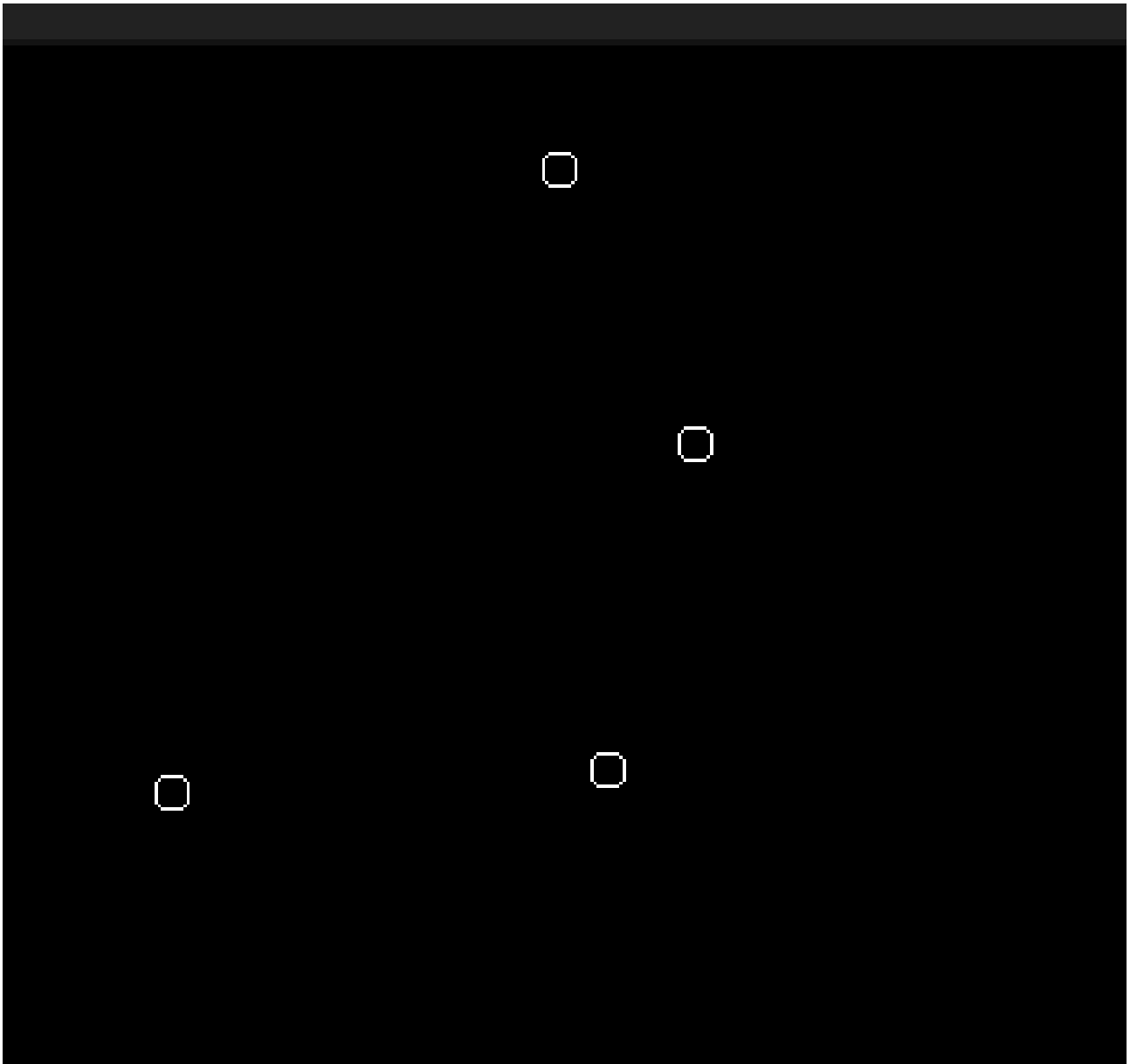


Рис. 1 — Исходная фигура

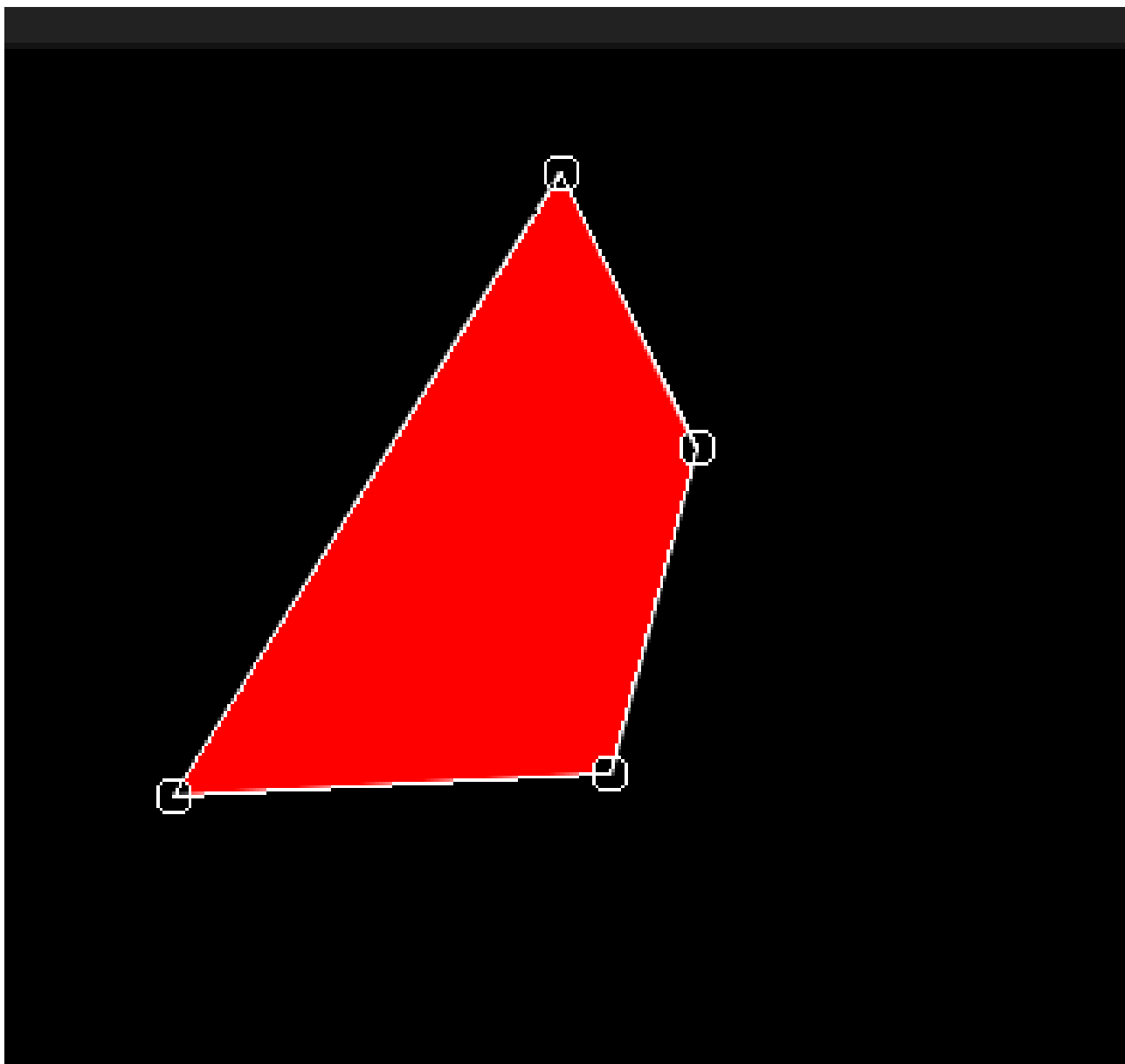


Рис. 2 — Фигура после заливки

Вывод

В ходе лабораторной работы мною были изучены принципы растровой графики и реализованы алгоритмы: целочисленный алгоритм Брезенхема с устранением ступенчатости и алгоритм построчной заливки с затравкой. Мною также были получены практические навыки работы с OpenGL, а также навыки реализации пользовательского ввода с помощью GLFW и алгоритмов растеризации.